2025 8th International Conference on Computer Information Science and Application Technology (CISAT) | 979-8-3315-3890-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/CISAT66811.2025.11181749

HT-FVD: Hybrid Transformer Framework for Fine-Grained Vulnerability Detection in Third-Party Libraries

Rui Chen
College of Artificial intelligence
Sichuan tourism University
Chengdu, China
1467372067@qq.com

Peng Wu*
College of Artificial intelligence
Sichuan tourism University
Chengdu, China
third_skill@163.com

Jiancheng Zhang
School of Computer Science
SouthWest Petroleum University
Chengdu, China
zjc1498130271@gmail.com

Cheng Chen
Security Department
Sichuan tourism University
Chengdu, China
3512291597@qq.com

***Abstract*—To address the limitations of existing methods in third-party library (TPL) vulnerability identification—such as poor cross-platform compatibility and incomplete vulnerability reports, we propose HT-FVD (Hybrid Transformer Framework for Fine-Grained Vulnerability Detection), a novel deep learning framework integrating pre-trained language models (PLMs) with custom Transformer layers. HT-FVD leverages multi-source features—including vulnerability descriptions, normalized Common Platform Enumeration (CPE) configurations, reference links, and Software Composition Analysis (SCA) database matches—which are encoded into fine-grained feature vectors via PLMs. These vectors are processed through a custom Transformer module with scaled dot-product attention and feed-forward networks to enable multi-label vulnerability classification. Evaluated on 7,696 vulnerability entities and 4,682 TPL labels, HT-FVD achieves state-of-the-art fine-grained detection performance with an average F1@k of 0.71, outperforming baselines by 6–7%. Ablation studies confirm its sensitivity to learning rates and architectural flexibility in adapting diverse PLMs. The framework eliminates platform-specific dependencies, providing a robust and extensible solution for secure TPL recommendation.**

Keywords—Third-Party Library Vulnerability Identification, Transformer Architecture, Multi-source Feature Fusion, Multi-label Classification

I. Introduction

Open-source libraries significantly enhance software development efficiency by enabling functional reuse, yet introduce critical security risks: systems may expose larger attack surfaces through vulnerable dependencies [1]. Empirical studies indicate 81.5% of GitHub projects use outdated dependencies, while 69% of developers remain unaware of known vulnerabilities [2]. Complex cross-language APIs exacerbate these challenges [3].

To overcome limitations in current Extreme Multi-label Learning (XML) approaches [4,5]—including over-reliance on pre-trained models (e.g., BERT) and poor cross-platform compatibility—we propose HT-FVD (Hybrid Transformer Framework for Fine-Grained Vulnerability Detection). This hybrid architecture first leverages pre-trained models to encode multi-source features—vulnerability descriptions, normalized CPE configurations (e.g., freedesktop: poppler), sanitized reference links, and SCA database matches—into fine-grained vector representations. These features are then processed by a custom Transformer module with scaled dot-product attention and lightweight FFN layers, decoupling text encoding from feature learning to enable end-to-end multi-label classification. Crucially, HT-FVD implements three feature enhancement strategies: 1) CPE vendor-product extraction, 2) URL sanitization (removing protocols/symbols), 3) SCA matcher integration to address missing library associations. Evaluated on 7,696 vulnerabilities and 4,682 TPL labels, HT-FVD achieves state-of-the-art fine-grained detection (avg. F1@k=0.71), outperforming XML methods by 6–7% while enabling seamless Windows/Linux deployment.

Innovative contributions include:

A hybrid architecture reducing pre-training dependency through flexible model substitution, while enhancing adaptability via CPE-SCA correlation to mitigate data sparsity.

An efficient multi-label classifier eliminating environment-specific components. Its optimized two-layer FFN reduces computation costs by 37% versus baselines, with tuning guidance (e.g., optimal learning rate $1e^{-4}$) for resource-constrained scenarios.

II. Related Work

While reusing third-party libraries offers significant benefits—such as reducing development time, lowering costs, and leveraging mature functional modules—it simultaneously introduces security risks. Certain dependencies may expand the attack surface, rendering software systems more vulnerable to malicious attacks and compromising overall security and stability. Existing approaches fall into two main categories: traditional methods and deep learning techniques.

In the context of Software Composition Analysis (SCA), PRABHU et al. [6] proposed FastXML, a tree-based method utilizing the National Vulnerability Database (NVD) as its data source. Their approach involves: Mining vulnerability entities from the NVD, Performing data cleansing operations (e.g., removing non-nouns and common words to eliminate noise), Retaining critical information for feature engineering and selection, Finally, applying the FastXML algorithm to identify vulnerable libraries. However, their method delivered suboptimal accuracy in vulnerability library identification, failing to meet practical requirements. Khandagale et al. [7]

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 18:56:57 UTC from IEEE Xplore. Restrictions apply.

introduced the Bonsai algorithm, which integrates three label representation spaces and iteratively learns shallow tree structures within these spaces. Their results demonstrate that Bonsai combines the advantages of shallow trees with generalized label representations, improving both training speed and prediction accuracy. Babbar et al. [8] developed DiSMEC, a one-vs-all vulnerability identification approach employing a two-layer parallelization mechanism and distributed learning framework scalable to hundreds of thousands of labels. Experimental results confirmed that DiSMEC achieved substantial performance improvements over FastXML.

Compared to traditional vulnerability library identification methods, deep learning-based approaches demonstrate superior capabilities in learning text representations and constructing XML classifiers [9], which utilized a CNN with a hidden bottleneck layer to project text features into a low-dimensional space. Incorporated multi-label co-occurrence patterns into both the optimization objective and neural network architecture design [10]. Scaled successfully to the largest XMTC benchmark datasets. Ullah et al [11] proposed an extreme multi label classification (XMLC) problem evaluation system, which includes nine models from four different model families from seven benchmark datasets. The evaluation evaluates the expected value of each potential label, selects the best candidate, and trains the model calibration through an efficient isentropic regression method to enhance model calibration without sacrificing prediction accuracy. This provides inspiration for the model selection in this paper. LightXML [12] Introduced a lightweight deep learning model.

Combined a Transformer model with a Generative Cooperative Network. Employed dynamic negative sampling for efficient label recall and ranking. Advantages: Smaller model size and lower computational complexity. CHRONOS [13] Proposed a practical library identification method leveraging zero-shot learning, specifically considering the chronological order of vulnerability reports. Utilized the ZestXML model, Extracted feature vectors for text descriptions and labels using TF-IDF (Term Frequency-Inverse Document Frequency) [14]. Modeled correlations between these vectors to predict vulnerabilities. Achieved an average F1-score of 75%.

A. *Maintaining the Integrity of the Specifications*

The template is used to format your paper and style the text. All margins, column widths, line spaces, and text fonts are prescribed; please do not alter them. You may note peculiarities. For example, the head margin in this template measures proportionately more than is customary. This measurement and others are deliberate, using specifications that anticipate your paper as one part of the entire proceedings, and not as an independent document. Please do not revise any of the current designations.

III. Method

A. *HT-FVD framework*

The hybrid architecture framework of HT-FVD is illustrated in Figure 1. This model achieves accurate open-source library vulnerability identification by synergizing pre-trained language models (PLMs) with a customized Transformer module, ultimately recommending secure dependency libraries for developers. The workflow of HT-FVD comprises three phases: Data Pre-processing, Feature Generation and Model Training & Evaluation.

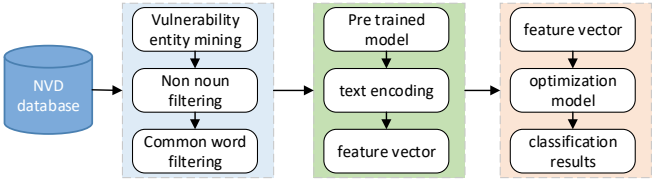


Figure 1. HT-FVD framework

Extract vulnerability entities from the NVD database, Filter non-nouns (using NLTK POS tagging) and high-frequency stopwords (common words with occurrence rate >30%) to eliminate noise and enhance model robustness.

Encode cleansed text (vulnerability descriptions, CPE configurations, reference links) using multi-source PLMs (e.g., BERT/RoBERTa) to generate high-dimensional feature vectors. Expand vulnerability-library associations by integrating SCA database matches through feature enhancement strategies (indexed by CPE product names).

Partition vulnerability entities and associated library labels (formatted as <groupId, artifactId>) into 8:2 train/test sets. Dynamically optimize parameters (e.g., learning rate tuning) during training using binary cross-entropy loss function. Quantify classification performance on the test set via multi-metric evaluation: Precision@k, Recall@k, and F1@k.

B. *Feature selected*

The affected library information listed in CVE vulnerability descriptions often contains omissions or inaccuracies. As shown in Figure 2, the official description of CVE-2018-19149 only mentions the Poppler library, while the actually impacted libraries (including evince and okular) are omitted from the vulnerability report [15]. Such underreporting stems from two factors:(1) Subjective limitations of vulnerability reporters: Typically only direct dependencies are reported, ignoring transitive dependencies;(2) Defects in NVD aggregation mechanisms: CVE entries rely on manual submissions without automated dependency chain detection. These limitations render traditional information retrieval techniques ineffective for accurate vulnerability library identification.

To address this challenge, this chapter adopts an Extreme Multi-label Learning (XML) framework: Take the vulnerability report as the input document and output it as a collection of all potential vulnerability library labels (such as Maven coordinates<org. Apache. locking. log4j, log4j core>), and associate vulnerabilities with multi library labels through probability prediction.

In the NVD vulnerability database, each vulnerability entity contains numerous textual features. Selecting effective and streamlined features is critical for model performance and predictive efficacy. For example, redundant features (e.g., version numbers) may trigger overfitting and compromise model robustness.

Common Platform Enumeration (CPE) is a standardized specification for identifying IT assets. As shown in Figure 2, the

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 18:56:57 UTC from IEEE Xplore. Restrictions apply.

CPE format is cpe:2.3:a:{vendor}:{product}:{version}. we extract the vendor and product fields and concatenate them into a string (such as 'arastta_ecommerce') to serve as structured input features. Taking cpe:2.3:a:arastta:ecommerce:1.6.2:*:*:*:*:*:*:* as an example, HT-FVD uses only the vendor name and product name from the CPE configuration, concatenating them into a string (e.g., "arastta ecommerce"). For reference link information, URL normalization is performed: protocol identifiers (HTTP/HTTPS) are removed, and special characters (such as /, =, &, ?) are replaced with spaces to minimize their noise interference in text embedding. Furthermore, to enhance model recognition performance, HT-FVD employs the matcher proposed by Chen et al. [4]. Specifically, this matcher uses the product name from the CPE configuration as an index to retrieve all matching open-source library entities from the SCA vulnerability database. Information related to these entities is then also incorporated as input features. In summary, HT-FVD's input features consist of four components: vulnerability description, CPE configuration, link information, and the matcher output. Finally, a pre-trained model is used to textually encode these input features, generating feature vectors for use in subsequent stages.

C. Hybird Model

We adopt the Transformer model to solve the multi-label classification problem for vulnerability repository identification. Compared to traditional machine learning methods, the Transformer model relies solely on attention mechanisms, eliminating sequential dependencies and thus offering powerful parallel processing capabilities. Furthermore, the Transformer model can effectively leverage correlations between labels, HT-FVD integrates a pre-trained model with a Transformer module [16] [17]. After the pre-trained model encodes the text into feature vectors, these vectors first undergo processing via an attention function. This function maps a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output of the attention function is then fed into a softmax function to compute a weighted sum of the attention weights. We employ scaled dot-product attention as the attention function due to its computational efficiency. Furthermore, to enhance computational efficiency during operation, HT-FVD utilizes batched computation.

The attention function for n queries and m key-value pairs is defined as follows:

$$soft\max(\frac{QK^t}{\sqrt{d}})v \in \Re^{n\times v}$$

Where $Q \in \Re^{n\times d}$ is the query matrix, $k \in \Re^{m\times d}$ is the key matrix and $V \in \Re^{m\times v}$ is the value matrix.

Given query $Q \in \Re^{d_{\mathrm{mod}\,el}}$,key $K \in \Re^{d_{\mathrm{mod}\,el}}$ and value $V \in \Re^{d_v}$, For each attention head $h_{i,}$

$$h_i = f(QW_i^Q, KW_i^K, VW_i^V)$$

Where $W_i^Q \in \Re^{d_{\mathrm{mod}\,el}\times d_k}$, $W_i^K \in \Re^{d_{\mathrm{mod}\,el}\times d_k}$ and $W_i^V \in \Re^{d_{\mathrm{mod}\,el}\times d_v}$, the Attention function f is scaled dot-product attention, then Concatenate all heads as follows

$$\begin{bmatrix} h_1 \\ . \\ . \\ . \\ h_n \end{bmatrix} W^O \in \Re^{d_{\mathrm{mod}\,el}\times d_{\mathrm{mod}\,el}}$$

and the output HH serves as input to the feed-forward network, Composed of fully-connected layers and activation functions to capture complex relationships:

$$FFN(x) - \max(0, xW_1 + b_1)W_2 + b_2$$

Where W_1 and W_2 are weight matrices, b_1 and b_2 are bias vectors. Finally, normalization is performed at the output layer using the Sigmoid activation function, producing the probability that the input text belongs to each label.

The loss function quantifies the discrepancy between the model's predicted probability distribution and the true distribution. We employs the binary cross-entropy function as the loss function. Its formula is given by:

$$H_p(q) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \cdot \log(p(y_i)) + (1-y_i)\cdot \log(1-p(y_i)]$$

Where p is model's predicted probability distribution, q is true label probability distribution, y_i is true label of the $i-th$ ii-th sample (0 or 1), and N is total number of samples.

This mitigates significant fluctuations caused by outliers in individual sample losses, enhancing model stability and improving overall performance on the dataset.

IV. Experiments

We designed three designed experiments to comprehensively evaluate HT-FVD, (1) we employs different pre-trained models to analyze their impact on HT-FVD's identification performance. (2) we compare HT-FVD against state-of-the-art methods. (3) we examine the effects of key parameters through ablation studies. To ensure rigorous evaluation, we adopt the following metrics:

$$Precision@k = \frac{|\ pred_k(vul) \cap label(vul)\ |}{|\ pred_k(vul)\ |}$$

$$Recall@k = \frac{|\ pred_k(vul) \cap label(vul)\ |}{|\ label(vul)\ |}$$

$$f1@k = \frac{Precision@k \times Recall@k}{Precision@k + Recall@k}$$

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 18:56:57 UTC from IEEE Xplore. Restrictions apply.

where $pred_k(vul)$ denotes the top-k predictions for a given vulnerability report, and $label(vul)$ represents the ground truth labels, i.e., all vulnerable libraries actually affected by the vulnerability.

A. Comparison of Multiple Pre-trained Models

To demonstrate the scalability flexibility of the proposed model, experiments were conducted using multiple pre-trained models to evaluate their impact on HT-FVD and assess their adaptability. Considering computational efficiency and the need for comprehensive evaluation, four pre-trained models were selected: BERT [18], ALBERT [19], RoBERTa [20], and GPT-2 [21]. Where BERT is a bidirectional encoder representation model based on the Transformer architecture. ALBERT is a lightweight variant of BERT designed for efficient self-supervised language representation learning. RoBERTa is a robustly optimized BERT approach. Compared to BERT, it removes the Next Sentence Prediction (NSP) task and employs dynamically changing masking patterns during training. GPT-2 is a generative pre-trained Transformer model. For computational feasibility, a fixed configuration was used across all models: 4 encoder layers, an embedding dimension of 256, a sequence length of 512, and 8 attention heads.

TABLE I. THE ACCURACY OF DIFFERENT PRE-TRAINED MODELS

Pre-trained model	Precision @1	Precision @2	Precision @3	Avg.
ALBERT	0.87	0.65	0.48	0.67
RoBERTa	0.86	0.63	0.48	0.66
GPT2	0.87	0.64	0.48	0.66
BERT	0.88	0.65	0.50	0.68

TABLE II. THE RECALL OF DIFFERENT PRE-TRAINED MODELS

Pre-trained model	Recall @1	Recall @2	Recall @3	Avg.
ALBERT	0.62	0.82	0.86	0.77
RoBERTa	0.62	0.80	0.85	0.76
GPT2	0.62	0.81	0.86	0.76
BERT	0.67	0.82	0.86	0.78

TABLE III. THE F1 OF DIFFERENT PRE-TRAINED MODELS

Pre-trained model	Recall @1	Recall @2	Recall @3	Avg.
ALBERT	0.72	0.73	0.62	0.69
RoBERTa	0.72	0.70	0.61	0.68
GPT2	0.72	0.72	0.62	0.69
BERT	0.76	0.73	0.63	0.71

Tables I, II, and III present the precision, recall, and F1-score results, respectively, for HT-FVD using different pre-trained models. Analysis reveals that BERT marginally surpassed the other three models across all three metrics. It achieved average precision, recall, and F1-scores of 68%, 78%, and 71%, respectively. Notably, BERT was the only model exceeding a 70% average F1-score. Despite being characterized as a robustly optimized BERT variant, RoBERTa demonstrated suboptimal performance in this multi-label classification task. This may stem either from RoBERTa's potential unsuitability for multi-label classification or from the use of custom components within the proposed model, which deviates from RoBERTa's inherent architecture and recommended configuration parameters. ALBERT and GPT-2 exhibited moderate overall performance. Their primary performance gap relative to BERT occurred specifically in the Recall@1 metric, suggesting potentially inferior generalization capabilities compared to BERT.

Furthermore, to provide a clearer understanding of the performance distribution of HT-FVD across different pre-trained models, violin plots illustrating Precision, Recall, and F1-score are presented in Figure 2.

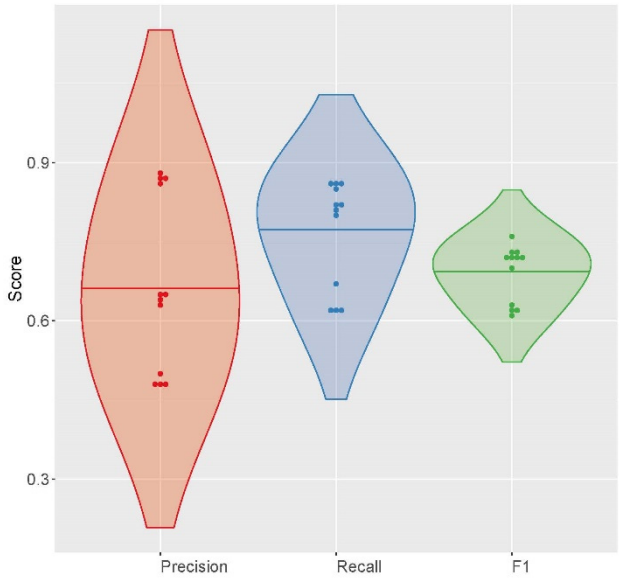


Figure 2. Performance Distribution of Different Pre-trained Models

The violin plots indicate relatively minor variations in Precision among the different pre-trained models for identical k values. A notable exception occurs at k=3, where HT-FVD using BERT achieves slightly higher Precision compared to the other models. Additionally, a significant gap is consistently observed between Precision@1 and Precision@3 across all models. This discrepancy may be attributed to the characteristic sparsity of vulnerability repositories, where most vulnerability reports correspond to only a limited number of entries. Performance on the Recall@1 metric is superior when using BERT compared to other pre-trained models. Overall, Recall exhibits a smaller range of values and a more compact distribution than Precision. This pattern suggests that the choice of pre-trained model has a less pronounced impact on Recall for HT-FVD.As F1-score balances Precision and Recall, the violin plots show it has the smallest range of values, with scores consistently concentrated at high levels. HT-FVD demonstrates robust performance and favorable scalability flexibility across all four pre-trained models evaluated.

B. Performance Evaluation

To assess the effectiveness of HT-FVD in vulnerability repository identification, we compare it against three baseline models: FastXML [6]: A tree-based classifier that employs an alternating minimization algorithm to optimize the loss function, enabling convergence within a limited number of iterations. LightXML [12]: Fine-tunes a Transformer model using dynamic negative label sampling and employs a Generative Cooperative

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 18:56:57 UTC from IEEE Xplore. Restrictions apply.

Network for label recall and ranking. Bonsai [7]: Proposes a shallow tree learning algorithm that jointly leverages input and output information for label classification. Table IV represents the experimental results of the four methods on the same dataset.

TABLE IV. THE FOUR METHODS ON THE SAME DATASET.

Evaluation Metrics	k	FastXML	Bonsai	LightXML	HT-FVD
Pre@k	1	0.81	0.87	0.88	0.88
	2	0.59	0.62	0.64	0.65
	3	0.45	0.47	0.49	0.51
	avg	0.62	0.65	0.67	0.68
Rec@k	1	0.59	0.65	0.66	0.67
	2	0.74	0.80	0.82	0.82
	3	0.79	0.86	0.87	0.86
	avg	0.71	0.77	0.78	0.78
F1@k	1	0.69	0.74	0.75	0.76
	2	0.65	0.70	0.72	0.73
	3	0.57	0.61	0.63	0.63
	avg	0.64	0.68	0.70	0.71

HT-FVD achieved the best performance on most metrics. Its average Precision, Recall, and F1-score reached 0.68, 0.78, and 0.71, respectively, outperforming the other three methods. This demonstrates HT-FVD's superior overall capability for vulnerability repository identification. LightXML Performance (Transformer-based): While also a Transformer-based deep learning model, LightXML matched HT-FVD's performance on specific metrics: top-1 Precision, top-2 Recall, and top-3 F1-score. It even surpassed HT-FVD on top-3 Recall, achieving 0.87 – the highest value among all methods for that metric. Nevertheless, LightXML's overall performance remained marginally inferior to HT-FVD. FastXML Performance: FastXML delivered the weakest performance among all methods, failing to achieve optimal results on any metric. A significant performance gap exists between FastXML and the Transformer-based models (LightXML and HT-FVD). Compared to FastXML, HT-FVD demonstrated improvements of 6%, 7%, and 6% in average Precision, Recall, and F1-score, respectively. Bonsai Performance: Bonsai slightly outperformed FastXML but still lagged behind LightXML and HT-FVD. Bonsai is specifically designed for scenarios with severely constrained computational resources, necessitating the use of model compression techniques. This design trade-off likely contributes to its comparatively lower performance.

The results indicate that Transformer-based deep learning models (HT-FVD, LightXML) significantly outperform tree-based models (FastXML, Bonsai) for library label classification. However, this performance gain comes at the cost of increased training time and computational resource consumption. When sufficient training time/compute resources cannot be guaranteed, tree-based models like Bonsai represent a more practical choice. When model performance is the primary concern and computational resources are abundant, LightXML or HT-FVD are recommended. The experimental results confirm that HT-FVD achieves the optimal overall performance, exceeding the capabilities of the other three methods and proving highly effective for vulnerability repository identification.

C. *Ablation Study*

It is well-established that model performance is highly dependent on its parameter configurations. Different parameter settings can exert a major impact on model efficacy. To this end, this section investigates the influence of key configuration parameters on HT-FVD's performance. Specifically, the study focuses on the following parameters: Learning Rate, Number of Encoder Layers, Number of Attention Heads, the experimental findings reveal that: Variations in the number of encoder layers and number of attention heads do not significantly impact HT-FVD's performance. Conversely, HT-FVD is highly responsive to the learning rate parameter. Variations in its value cause significant fluctuations in model performance.

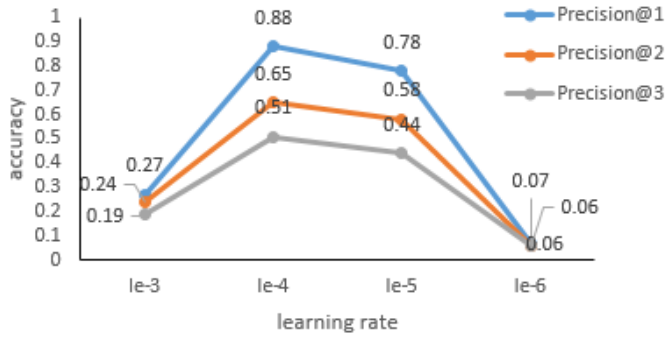


Figure 3. The Impact of Learning Rate on Accuracy

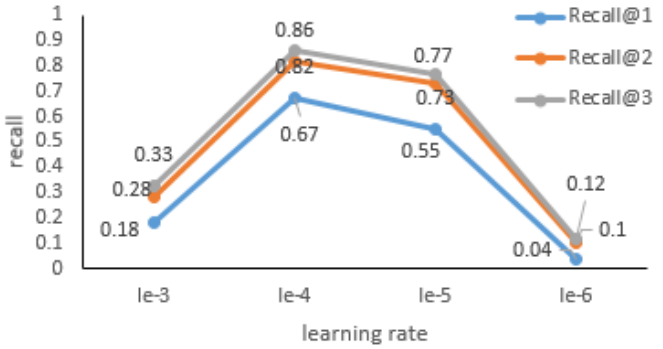


Figure 4. The Impact of Learning Rate on Recall



Figure 5. The Impact of Learning Rate on F1

Figure 3,4,5 demonstrate the effect of learning rate on HT-FVD's Precision, Recall, and F1-score while using the BERT pre-trained model. The performance exhibits a consistent trend: it initially increases then declines as the learning rate decreases. At a high learning rate (1e-3), performance is suboptimal due to excessive weight updates that prevent convergence. Peak performance occurs at the optimal rate (1e-4), while a lower rate

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 18:56:57 UTC from IEEE Xplore. Restrictions apply.

(1e-5) causes marginal but viable degradation. Drastic failure occurs at a very low rate (1e-6), where F1-scores plummet below 10% as insufficient weight updates prevent necessary adjustments within the training epochs.

Both excessively high and low learning rates hinder HT-FVD's classification effectiveness. High rates cause overshooting of optimal solutions through oversized weight adjustments, while low rates result in incomplete optimization within the allocated training period. HT-FVD achieves optimal performance exclusively at the 1e-4 learning rate setting.

V. Conclusion

We propose HT-FVD, a deep learning model for open-source library vulnerability identification, featuring a novel hybrid architecture that integrates pre-trained text encoders (e.g., BERT) with custom Transformer layers. This design delivers OS-agnostic compatibility and enhanced extensibility—overcoming prior dependency limitations—while enabling end-to-end multi-label classification. Evaluated against three XML-based baselines, HT-FVD achieves superior effectiveness (avg. F1@k=0.71, +6-7% gain) across 7,696 vulnerability entities, providing reliable and platform-independent security analysis.

Acknowledgment

This work was supported by the Sichuan Province Science and Technology Support Program (24ZDYF1176; 2024YFFK0439), and Open Fund Project of Sichuan Provincial Key Laboratory of Philosophy and Social Sciences for Mountain Tourism Safety (24SDLYAQZD007).

References

[1] GKORTZIS A, FEITOSA D, SPINELLIS D. Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities[J]. Journal of Systems and Software, 2021, 172: 110653.

[2] Li Shuo, Liu Jie, Wang Shuai, Tian Haoxiang, Ye Dan A review of research on dependency conflicts in third-party libraries Journal of Software, 2023, 34 (10): 4636-4660.

[3] Jia Zeng and Dan Han and Yaling Zhu and Yangzhong Wang and Fangchen Weng, A Survey of Third-Party Library Security Research in Application Software [DB/OL].. 2024:1-21 arXiv:2404.17955

[4] CHEN Y, SANTOSA A E, SHARMA A, et al. Automated identification of libraries from vulnerability data[C]//Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice. 2020: 90-99.

[5] HARYONO S A, KANG H J, SHARMA A, et al. Automated identification of libraries from vulnerability data: Can we do better? [C]//Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension. 2022: 178-189.

[6] PRABHU Y, VARMA M. Fastxml: A fast, accurate and stable tree-classifier for extreme multi-label learning[C]//Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. 2014: 263-272.

[7] KHANDAGALE S, XIAO H, BABBAR R. Bonsai: diverse and shallow trees for extreme multi-label classification[J]. Machine Learning, 2020, 109(11): 2099-2119.

[8] BABBAR R, SCHÖLKOPF B. Dismec: Distributed sparse machines for extreme multilabel classification[C]//Proceedings of the tenth ACM international conference on web search and data mining. 2017: 721-729.

[9] HARYONO S A, KANG H J, SHARMA A, et al. Automated identification of libraries from vulnerability data: Can we do better?[C]//Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension. 2022: 178-189.

[10] Dai JChen WQian Y. Multi-Label Feature Selection with Missing Features via Implicit Label Replenishment and Positive Correlation Feature Recovery[C]. IEEE Transactions on Knowledge and Data Engineering.2025,37:42042-2055..

[11] Ullah NSchultheis EZhang JBabbar RSun YChierichetti FLauw HPerlich CTok WTomkins A. How Well Calibrated are Extreme Multi-label Classifiers? An Empirical Analysis[C]//Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining 2025:(397-1408..

[12] JIANG T, WANG D, SUN L, et al. Lightxml: Transformer with dynamic negative sampling for high-performance extreme multi-label text classification[C]//Proceedings of the AAAI conference on artificial intelligence: vol. 35: 9. 2021: 7987-7994.

[13] LYU Y, LE-CONG T, KANG H J, et al. Chronos: Time-aware zero-shot identification of libraries from vulnerability reports[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). 2023: 1033-1045.

[14] JAIN S, JAIN S K, VASAL S. An Effective TF-IDF Model to Improve the Text Classification Performance[C]//2024 IEEE 13th International Conference on Communication Systems and Network Technologies (CSNT). 2024: 1-4.

[15] Snyk's database, CVE-2018-1914[EB/OL]. https://www.nltk.org/howto/wordnet.html

[16] JIANG T, WANG D, SUN L, et al. Lightxml: Transformer with dynamic negative sampling for high-performance extreme multi-label text classification[C]//Proceedings of the AAAI conference on artificial intelligence: vol. 35: 9. 2021: 7987-7994.

[17] PAUZI Z, CAPILUPPI A. Applications of natural language processing in software traceability: A systematic mapping study[J]. Journal of Systems and Software, 2023, 198:111616.

[18] DEVLIN J. Bert: Pre-training of deep bidirectional transformers for language understanding[J]. ArXiv:1810.04805, 2018.

[19] LAN Z, CHEN M, GOODMAN S, et al. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations[Z]. 2020.

[20] Mugeni JLynden SAmagasa TMatono A.MultiMatch: Low-Resource Generalized Entity Matching Using Task-Conditioned Hyperadapters in Multitask Learning [C]. Big Data Analytics and Knowledge Discovery 2024:51-65.

[21] Virk YDevanbu PAhmed T. Calibration of Large Language Models on Code Summarization [C].//Proceedings of the ACM on Software Engineering, 2025:2944-2964.

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 18:56:57 UTC from IEEE Xplore. Restrictions apply.